ESPECIFICACIÓN TÉCNICA · BASE MULTIDIOMA 1.1.2

LinguaNova Multidioma Archivos nuevos y aplicación por fases

Alcance inmediato: español, francés, portugués, ruso e inglés.
Previsión de arquitectura: ampliación futura a 139 idiomas de destino.

> **Base de referencia:** LinguaNova_V2.2_Multidioma..zip, adjuntado en esta conversación. Sus 283 archivos se conservan íntegros. Las mejoras de la versión interna 1.1.2 forman parte de la base protegida y no se reemplazan por las de una copia anterior.

Qué define esta entrega

• Nombres y responsabilidades de los archivos añadidos, interfaces, formatos de datos, orden de aplicación y pruebas de aceptación.

• Cambio de idioma solo desde Preferencias o su equivalente, sin atajos ni paneles externos, conservando el estilo del juego y la elección tras reiniciar.

• Diagnóstico del porcentaje restante, perfiles del español y puente exclusivo entre OCR y traductor con destino único.

• Contratos preparados para crecer, manteniendo únicamente cinco idiomas base habilitados en esta etapa.

Estado

Documento de diseño técnico. No implementa los módulos descritos, no modifica el ZIP ni sus archivos y no certifica traducciones reales. Las funciones nuevas se incorporarán mediante código nuevo y copias independientes.

01 · BASE AUTORIZADA

La versión nueva es el punto de partida

El ZIP adjunto contiene la carpeta LinguaNova_Multidioma y declara VERSION = 1.1.2. Se verificaron 283 archivos, 86 archivos Python bajo src/ y 115 módulos de pruebas test_*.py. Los 86 fuentes se analizaron sintácticamente sin errores; no se ejecutó la aplicación ni su batería funcional.

Mejora recibida	Qué debe conservarse
Selector acumulativo	Generar otro idioma mantiene los ya instalados; no sustituye toda la lista por el último generado.
Integración con el juego	Reutiliza controles nativos reconocibles y dispone de selector alternativo con Shift+L cuando corresponde.
Cambio en caliente	Sigue la preferencia activa durante la partida y permite volver a Original.
Cachés del juego	Limpia los datos del idioma anterior; no carga una traducción previa al elegir Original o un destino ausente.
Protección de fuentes	Genera automáticamente zzz_linguanova_ocultar_fuentes.rpy; los recursos no aparecen como idiomas.
Compatibilidad tipográfica	Nuevas fuentes fuera de tl/, conservando fuentes y rutas antiguas y los bloques de otros idiomas.
Convivencia con el MOD	Evita duplicar selector o atajo cuando el MOD administra el panel; filtra las carpetas de recursos.
Guardado comprobado	Una escritura fallida no se presenta como integración exitosa ni destruye el auxiliar anterior.

Diferencia frente a la base del primer informe

La comparación con el RAR anterior identifica 259 archivos idénticos, 18 modificados y 6 añadidos; no se eliminaron archivos. Estos son cambios ya presentes en el ZIP entregado por el usuario, no modificaciones realizadas para este informe.

El informe anterior analizaba la versión interna 1.1.0 y 277 archivos. Esos datos no deben usarse como inventario de la nueva base. La implementación futura debe congelar y verificar esta versión 1.1.2 completa.

02 · PRESERVACIÓN

Archivos y comportamientos protegidos

Archivos existentes, sin editar	Invariante de conservación
src/core/language_menu_template.py	Conservar el fuente y la protección automática. Su salida heredada contiene Shift+L: el exportador nuevo no debe generar ese selector ni modificarlo después para retirar teclas.
src/core/pipeline/saving.py	Conservar la integración acumulativa, la detección de menús nativos y la comprobación del auxiliar escrito.
src/core/runtime_hook_template.py	Mantener la recarga por idioma activo, limpieza de datos y compatibilidad de los formatos JSON.
src/tools/font_injector.py	Mantener fuentes fuera de tl/ y conservación de bloques y rutas anteriores.
src/ui_mod/generator.py y resto de ui_mod/	Conservar su aislamiento, glosarios, correcciones manuales, exclusión de recursos y protección automática.
orchestrator.py, tl_parser.py, motores y guards	Mantener funciones, contadores, validaciones y rechazo de respuestas dañadas.
Main.qml, locales/, editor y herramientas	Mantener la interfaz y las utilidades recibidas. No ampliar sus listas mediante reemplazos de constantes.
Versionado, actualizador, compilación y pruebas	Conservar versión 1.1.2, repositorio de actualización, configuración original y pruebas históricas.

Regla verificable

Crear un manifiesto inicial con ruta relativa, tamaño y SHA-256 de cada uno de los 283 archivos. Repetir la comparación tras cada fase y antes de empaquetar: cero cambios, eliminaciones o renombrados en ese conjunto. Los archivos nuevos se enumeran por separado.

Trabajo siempre sobre copias

Conservar el ZIP de referencia sin ejecución. Extraer una copia de desarrollo y usar una copia independiente del juego. Los registros, cachés y salidas nuevas tienen su propio espacio; las rutinas existentes que escriban solo se invocan sobre staging.

La ampliación reutiliza la protección automática de recursos sin un segundo filtro ni parche manual. Conservar el código recibido no obliga a activar todos sus comportamientos en la salida nueva: los generadores antiguos que añaden atajos se conservan como referencia y no se usan para crear la interfaz final solicitada.

03 · DOS CAPACIDADES DISTINTAS

Cinco destinos de creación y selector del juego

> **Alcance vigente:** cinco idiomas base habilitados: español, francés, portugués, ruso e inglés. La posibilidad de mostrar otros idiomas ya instalados en un juego se conserva; no significa que el traductor pueda generarlos en esta etapa.

Componente	Situación y aplicación
Menú de destino de LinguaNova	La lista actual sigue siendo de cinco idiomas. El panel nuevo de trabajos usa esos mismos cinco, con un perfil adicional al seleccionar español.
Selector dentro del juego	La base ya descubre idiomas instalados. La salida nueva usa esa información dentro de Preferencias, conserva su estilo y no añade atajos, paneles o botones flotantes.
Variante de español	LATAM y España son perfiles del idioma español. No se presentan como dos idiomas base adicionales en el catálogo.
Modelo OCR de origen	Se selecciona para leer el texto que contiene la imagen. No se configura por el idioma al que se quiere traducir.
Destino del trabajo	Se selecciona una sola vez y acompaña texto e imágenes. El puente OCR / traductor recibe esa decisión.

Añadir funciones sin editar el menú original

El código nuevo dispone de run_next.py y un panel complementario de trabajos. La aplicación clásica permanece disponible con sus botones y sus cinco destinos. No se afirma que sus botones antiguos adquieran funciones OCR sin una conexión nueva.

Los trabajos coordinados se crean en el panel nuevo de LinguaNova. La selección de su destino gobierna texto y OCR. Este panel no se instala como interfaz del juego: allí los idiomas deben integrarse en sus controles y estilos originales, conservando la navegación. El juego cambia traducciones ya instaladas y no envía solicitudes al OCR durante la partida.

Dependencias

El panel conoce servicios de trabajo; estos conocen contratos y adaptadores. Los proveedores y los auxiliares del juego no dependen del panel. Los adaptadores invocan funciones públicas compatibles sin sustituir sus implementaciones.

04 · ARRANQUE Y CONTRATOS

Archivos nuevos para la base de la ampliación

Todos los nombres siguientes son propuestas. El paquete linguanova_next se agrega junto a src/ dentro de la copia del proyecto. Su nombre distingue la ampliación y no cambia la marca del producto.

Archivo nuevo propuesto	Responsabilidad / contrato
run_next.py	Arranque nuevo. Crea servicios y el panel complementario; permite abrir el modo clásico. No modifica run.py ni importa una función para reemplazarla.
linguanova_next/ bootstrap.py	Inicializa dependencias, rutas y proveedores mediante composición explícita. Comprueba compatibilidad con la base bloqueada.
linguanova_next/ baseline.py	Genera y verifica el inventario protegido. API propuesta: capture_baseline() y verify_baseline(). No actualiza un hash discrepante como si fuera válido.
linguanova_next/contracts/ job.py	TranslationJob inmutable: proyecto, origen, idioma base, locale, carpeta Ren’Py, perfil y versiones.
linguanova_next/contracts/ entry.py	Occurrence y Candidate: ID estable de entrada, ubicación, contexto, original y candidata. Separa ocurrencias de textos únicos.
linguanova_next/contracts/ result.py	ValidationResult y EntryResult: controles, causas, escritura, procedencia y revisión.
linguanova_next/contracts/ events.py	Eventos versionados de progreso, errores y revisión; siempre con job_id, request_id y recurso cuando aplique.
linguanova_next/core/ translation_job.py	Autoridad del trabajo y sus etapas. Coordina cancelación y reanudación; no ejecuta otra cola paralela oculta.
linguanova_next/core/ work_queue.py	Adaptación o reutilización explícita de la cola existente para la sesión nueva. Limita la concurrencia por tipo de subtarea.
linguanova_next/core/ translation_report.py	Unifica vistas del inventario, resultados y métricas sin recontar lo mismo por cada evento.

Inicializadores y reglas de importación

Añadir __init__.py en los paquetes necesarios, con importaciones ligeras. Importar contracts o baseline no debe cargar Qt, GPU, un modelo OCR ni un cliente remoto. Las dependencias opcionales se cargan al activar su proveedor.

La ejecución nueva puede reutilizar una clase de la base mediante su constructor y métodos disponibles. No se usa monkey-patching, herencia que sustituya métodos de la base, edición en tiempo de ejecución ni transformaciones del código durante la compilación.

05 · IDIOMAS Y PERFILES

Archivos nuevos con cinco entradas iniciales

Archivo nuevo propuesto	Responsabilidad / contrato
linguanova_next/core/ language_registry.py	Resuelve identidades y capacidades. list_destinations() devuelve los cinco idiomas habilitados; resolve_profile() conserva la variante.
linguanova_next/data/ languages.json	Datos iniciales para es, fr, pt, ru y en. Incluir schema_version y catalog_version; no añadir 134 destinos ficticios.
linguanova_next/data/ provider_capabilities.json	Códigos, límites y evidencia por proveedor/modelo. Estados: compatible comprobado, no compatible o no verificado.
linguanova_next/data/profiles/ es_419.json	Perfil de español latinoamericano neutro, sin modismos ni usos propios de España.
linguanova_next/data/profiles/ es_ES.json	Perfil de español estándar de España, sin modismos ni usos propios de Latinoamérica.
linguanova_next/linguistic/ profile_validator.py	Evalúa el perfil por contexto. Devuelve razones y nivel de verificación; no certifica una variante solo porque un detector devuelva es.
linguanova_next/linguistic/ prompt_policy.py	Construye instrucciones del perfil para solicitudes simples, lotes, reintentos y motores alternativos. No altera los prompts globales de la base.
linguanova_next/adapters/ provider_adapter.py	Convierte el contrato nuevo a TranslationRequest existente cuando sea compatible; conserva locale y perfil en el contexto externo y metadatos.
linguanova_next/adapters/ config_view.py	Vista propia e inmutable para las APIs reutilizadas. No edita ConfigManager ni fuerza idiomas desconocidos a inglés.

Identidad estable y compatibilidad

El trabajo distingue target_language_id = es, target_locale = es-419 y renpy_id = spanish_latam. Para España utiliza es-ES y spanish_spain. Las carpetas spanish ya existentes se conservan como destino heredado de variante no especificada hasta revisión.

Francés, portugués, ruso e inglés mantienen sus códigos y carpetas actuales. No se cambia su variante regional por iniciativa de este informe. La validación de fuente, origen automático y capacidades se realiza con los motores realmente disponibles.

Reglas de alias

es, spanish y Español no eligen automáticamente España o LATAM para datos antiguos. En trabajos nuevos, el usuario selecciona español y el perfil predeterminado propuesto es LATAM, visible y persistido. El código del proveedor se resuelve al enviar la petición; nunca se usa la carpeta Ren’Py como código de API.

06 · TEXTO, AUDITORÍA Y DIAGNÓSTICO

Archivos nuevos para explicar lo pendiente

Archivo nuevo propuesto	Responsabilidad / contrato
linguanova_next/adapters/ extraction_adapter.py	Consume las lecturas del parser y extracción profunda que sean compatibles. Preserva ubicación y crea un inventario de ocurrencias.
linguanova_next/core/ text_classification.py	Clasifica diálogo, narración, UI, vocalización, identificadores, rutas y casos inciertos según su contexto.
linguanova_next/core/ supplemental_extraction.py	Recoge candidatos omitidos cuando haga falta, con código nuevo y sin abrir los filtros técnicos de la base. Deduplica por ocurrencia.
linguanova_next/core/ coverage_audit.py	Auditoría de solo lectura: inventario frente a salidas, copias, vacíos, faltantes, duplicados, bloqueos y errores de escritura.
linguanova_next/core/ residual_review.py	Construye el listado de entradas restantes y las agrupa por causa; no las convierte por sí mismo en traducciones ni alias.
linguanova_next/core/ runtime_diagnostics.py	Administra diagnóstico opcional del creador. Solo observación compatible; no sustituye funciones del hook recibido.
linguanova_next/adapters/ diagnostics_adapter.py	Lee contadores e informes existentes y los vincula por ID. Evita sumar categorías que se solapan.
linguanova_next/storage/ result_store.py	Guarda candidatos, controles, decisiones y procedencia de la ruta nueva con escritura atómica y esquema versionado.

Salidas nuevas del diagnóstico

coverage_report.json presenta métricas y universos. unresolved_entries.jsonl enumera entrada, ubicación, motivo y acción pendiente. preserved_entries.jsonl registra conservaciones justificadas. review_actions.jsonl conserva decisiones y correcciones para volver al flujo normal de validación.

Qué no se repara automáticamente

Una copia del original puede ser un nombre correcto o una frase sin traducir. Una cadena corta puede ser diálogo o un identificador técnico. El módulo registra una decisión contextual y su evidencia. No cambia nombres o símbolos para aumentar el porcentaje ni elimina pendientes del denominador sin una política identificada.

La cobertura durante el juego se informa por rutas observadas. Una ruta sin punto de observación seguro queda no verificada. Los logs no se incorporan a la caché ni al paquete de jugadores.

07 · PUENTE OCR / TRADUCTOR

Archivos nuevos y comunicación exclusiva

El destino seleccionado en el menú del trabajo nuevo se fija antes de iniciar OCR. El canal entre reconocimiento y traducción es independiente de los hooks del juego y del MOD. No se traduce mediante OCR durante la partida.

Archivo nuevo propuesto	Responsabilidad / contrato
linguanova_next/bridge/ ocr_translation_bridge.py	Abre un canal por trabajo. Recibe regiones OCR, solicita traducción y correlaciona respuestas con el destino fijado.
linguanova_next/contracts/ ocr_messages.py	RegionRequest y RegionResponse: trabajo, imagen, región, origen estimado, destino, perfil, versión y estado.
linguanova_next/image/ ocr_engine.py	Interfaz del reconocedor. Detecta y transcribe origen; no decide el destino.
linguanova_next/image/ image_translation.py	Cliente del puente para las regiones. Usa el mismo servicio textual y glosario del trabajo.
linguanova_next/adapters/ ocr_provider.py	Adapta el reconocedor elegido, sus modelos y confianza a regiones normalizadas. Su implementación depende del OCR seleccionado.



Condiciones de aceptación

Cada mensaje incluye job_id, request_id, resource_id, region_id, image_hash, target_language_id, target_locale, profile_id y profile_version. Una respuesta sin correlación o de un trabajo cancelado no puede pasar al compositor.

El OCR configura sus modelos de lectura por el origen de la imagen. Una imagen japonesa se reconoce como entrada japonesa aunque el resultado elegido sea español LATAM. La identidad de destino viaja en el contrato; la comprobación lingüística evalúa el texto devuelto, no confía solo en sus etiquetas.

08 · IMAGEN, CACHÉS Y MODELOS

Separar generación, validación y recursos

Archivo nuevo propuesto	Responsabilidad / contrato
linguanova_next/image/ image_inventory.py	Inventario completo: hash, ruta, formato, dimensiones, transparencia y vínculo con atlas o animaciones; reportar formatos no soportados.
linguanova_next/image/ image_reconstruction.py	Restauración clásica dentro de máscaras, conservando el original. Interfaz de fondo restaurado y máscara efectiva.
linguanova_next/image/ generative_image_reconstruction.py	Proveedor opcional de inpainting local. Se añade después del flujo clásico; un fallo conserva el original.
linguanova_next/image/ image_compositor.py	Texto en capa separada, usando destino y perfil del resultado aceptado. Mide tipografía y desbordamientos.
linguanova_next/image/ image_validation.py	Valida formato, transparencia, regiones, texto residual, legibilidad y alteraciones fuera de máscara.
linguanova_next/image/ visual_coverage_audit.py	Distingue inventariada, sin texto detectable, incierta, pendiente, generada, revisada y fallida.
linguanova_next/models/ model_registry.py	Resuelve proveedores y modelos instalados. La ejecución básica no exige un modelo generativo.
linguanova_next/models/ model_manifest.py	Versiones, hashes, licencia, requisitos, activación e información para recuperar una versión compatible.
linguanova_next/storage/ cache_store.py	Namespaces separados para texto, MOD, OCR y composición; índices y coincidencias respetan destino, contexto y versiones.

Dependencias y uso de la caché

OCR puede reutilizarse entre destinos si imagen, reconocedor y parámetros coinciden. Una composición solo se reutiliza si coinciden traducción, destino, fuente, máscara y modelo. La caché de traducciones aceptadas nunca recibe rechazos ni observaciones de diagnóstico.

No se selecciona un modelo concreto por popularidad. Antes de incorporarlo se comprueban su tarea, licencia, requisitos y distribución. La opción generativa no sustituye la composición clásica ni implica rehacer toda la imagen.

La separación descrita se aplica primero a los cinco destinos actuales. Otros alfabetos y direcciones se incorporarán únicamente al ampliar el catálogo con su soporte comprobado.

09 · APLICACIÓN DE CÓDIGO

Cómo reutilizar la integración mejorada

El adaptador nuevo reutiliza el descubrimiento de idiomas y la protección de fuentes compatibles. La salida de Preferencias se genera desde código nuevo: el generador recibido siempre añade teclas de idioma y no se invoca para este paquete.

Archivo nuevo propuesto	Responsabilidad / contrato
linguanova_next/adapters/ language_menu_adapter.py	Lee idiomas instalados y contratos compatibles de la base. Deriva al exportador nuevo de Preferencias; no llama al generador heredado que siempre añade Shift+L.
linguanova_next/adapters/ preferences_source_adapter.py	Lee la pantalla original sin modificarla y valida su receta: contenedor, estilos y navegación. Un diseño no reconocido queda pendiente de integración.
linguanova_next/adapters/ runtime_export_adapter.py	Selecciona la salida nativa o el runtime estándar recibido según el trabajo. Conserva formatos y comportamiento de cambio de idioma.
linguanova_next/adapters/ mod_adapter.py	Lee candidatos y datos del MOD mediante contratos compatibles, preservando su aislamiento. No instala el selector heredado ni introduce el puente OCR en su hook.
linguanova_next/adapters/ font_adapter.py	Usa la protección e inyector existentes sobre staging. Comprueba rutas antiguas, fuentes rusas y conservación de otros bloques.
linguanova_next/packaging/ mod_profile_export.py	Salida MOD nueva para los cinco destinos, sin panel ni atajos de idioma. Admite perfiles mediante contratos nuevos; no edita el hook recibido ni transforma su texto generado.

Ejemplo propuesto para el flujo nuevo, no implementado

```
from src.core.language_menu_template import (
    installed_language_folders, install_resource_filter,
)

stage = str(job.staging_game_dir)
install_resource_filter(stage)
result = preferences_exporter.build(
    game_dir=stage, installed_languages=installed_language_folders(stage),
    ui_profile=verified_game_ui_profile,
    add_language_hotkeys=False, add_overlay=False,
    persist_choice=True,
)
native_ui_policy.require_preferences_only(result)
```

preferences_exporter y native_ui_policy son servicios nuevos. Su salida se limita a los controles de Preferencias y la persistencia. La protección de fuentes recibida se reutiliza sin modificarla; no se usa render_language_picker() ni create_language_menu_integration(), porque sus auxiliares actuales incluyen teclas de idioma.

El ejemplo define contratos futuros. La salida será un auxiliar nuevo en staging, sin editar screens.rpy ni posprocesar el selector antiguo.

10 · PORCENTAJE Y PENDIENTES

Diagnosticar el 100 % con datos por entrada

La nueva base conserva la corrección que diferencia copias del origen y entradas vacías. No se plantea deshacerla. El análisis nuevo debe explicar por qué quedan entradas sin cambiar y qué parte requiere traducción.

Valores citados en la documentación recibida	Español	Portugués
Entradas consideradas	74.460	74.460
Líneas cambiadas	70.665	70.314
Proporción de líneas cambiadas	94,9 %	94,4 %
Iguales al original	3.795	4.146
Salidas bloqueadas registradas	915	601

Son cifras reproducidas en docs/VERIFICACION_MULTIDIOMA.md. No se recibió aquí el diagnóstico original de aquella ejecución para clasificar cada entrada. Los bloqueos pueden estar incluidos entre las líneas sin cambiar; no se suman como si fueran grupos disjuntos.

Procedimiento del auditor

• Leer inventario, traducciones escritas y diagnósticos de la misma ejecución, idioma y variante. Identificar ocurrencias, textos únicos y fallos por separado.

• Para cada resto, registrar si es conservación válida, respuesta idéntica, bloqueo de sintaxis, fallo de motor, error de escritura, omisión o incertidumbre.

• Separar candidatas rechazadas de las salidas realmente escritas. Conservar ubicaciones y motivos; no convertir todo lo restante en sonidos o nombres.

Métricas distintas

Procesamiento: tareas terminadas / tareas planificadas. Cambio textual: entradas distintas del origen / entradas leídas. Traducción validada: traducciones aceptadas / entradas traducibles detectadas. Resolución del inventario: traducciones aceptadas más conservaciones aprobadas / universo textual auditado.

Una resolución del inventario al 100 % puede incluir nombres conservados y debe mostrarlo. No se etiqueta como todas las líneas traducidas. La cobertura de imágenes y las rutas de juego no visitadas se informan aparte.

11 · ESPAÑOL Y MOTORES

La variante es una propiedad del trabajo

> **Regla fija:** español LATAM neutro sin modismos ni usos propios de España; español de España estándar sin modismos ni usos propios de Latinoamérica. El vocabulario compartido es correcto en ambos perfiles.

Aplicación sobre cinco idiomas

La lista de idiomas base contiene una única entrada Español. Un control de perfil permite LATAM o España. La variante se guarda aparte y llega a texto, MOD, OCR y composición de imágenes. Las demás cuatro lenguas conservan su configuración actual.

Punto de control	Aplicación en código nuevo
Proveedor	Resolver el código regional que admita el servicio y modelo reales. No suponer que todos aceptan es-419 o es-ES.
Instrucciones IA	Agregar perfil explícito en solicitudes simples, lotes y reintentos; las preferencias personalizadas no pueden anularlo dentro del trabajo nuevo.
Respuesta	Comprobar estructura, idioma y usos regionales en contexto; detectar español genérico no valida automáticamente LATAM.
Reintento y fallback	Conservar destino y perfil, registrar motor efectivo y repetir controles. Un motor alternativo no cambia de variante.
Rechazo	Guardar motivo en diagnóstico. No publicar ni cachear como traducción aceptada una infracción detectada.
Revisión	Separar aprobación automática de aprobación lingüística. La publicación bajo la exigencia editorial estricta requiere revisión del material exportado.

Motores conservados

No se elimina ni se reintroduce un motor como parte del plan. Se conservan los presentes en la base y sus configuraciones. DeepSeek sigue su ruta compatible con OpenAI donde corresponda. El modo pseudo es de prueba, no una traducción certificada. Las restricciones del MOD no se equiparan automáticamente a las del traductor principal.

Datos heredados

Las entradas es o spanish existentes no se renombran como LATAM ni España. Se conservan las correcciones manuales y su procedencia, con variante desconocida hasta revisión. Ningún índice rápido o coincidencia entre motores puede mezclar los perfiles nuevos.

12 · FORMATO DE TRABAJOS

Contratos versionados y eventos de progreso

Ejemplo de estructura propuesta para un trabajo nuevo. Los nombres muestran separación entre idioma base, variante y salida de Ren’Py; no son datos de una ejecución real.

```
{
  "schema_version": 1,
  "job_id": "job-001",
  "project_id": "game-001",
  "base_manifest_hash": "<huella de la base 1.1.2>",
  "target_language_id": "es",
  "target_locale": "es-419",
  "renpy_id": "spanish_latam",
  "profile_id": "latam_neutro_sin_modismos",
  "profile_version": "1",
  "catalog_version": "1",
  "glossary_hash": "<hash>",
  "rules_hash": "<hash>"
}
```

Eventos mínimos

JobCreated, ExtractionCompleted, OcrRegionDetected, TranslationCandidate, ValidationRejected, EntryApproved, ResourceWritten, JobPaused y JobCompleted. Todos identifican trabajo y secuencia; los de recurso incluyen request_id, occurrence_id o region_id. El nombre JobCompleted no implica cobertura lingüística total.

Escritura y reanudación

Guardar el contrato y las versiones antes de procesar. Registrar resultados atómicamente. Al reanudar, comparar imagen o texto fuente, perfil, catálogo y glosario. Los cambios relevantes crean una nueva revisión o tarea y no reutilizan sin más un resultado antiguo.

Cambio de selección y cancelación

Modificar el selector mientras se trabaja prepara otro trabajo. El destino activo no se cambia dentro de los mensajes pendientes. La cancelación invalida respuestas tardías, y las operaciones de composición solo aceptan resultados de la revisión vigente.

Los datos operativos se guardan fuera del árbol protegido: settings_next.json, jobs/<job_id>/job.json, resultados y cachés por namespace. No se incluyen credenciales ni rutas locales privadas en el paquete final.

13 · ARCHIVOS DE SALIDA

Qué se crea y qué se mantiene en el juego

Archivo nuevo propuesto	Responsabilidad / contrato
linguanova_next/packaging/ package_manifest.py	Relaciona archivos exportados con hashes, destino, perfil, estado, generador y versiones. Define propiedad de cada salida.
linguanova_next/packaging/ package_validator.py	Comprueba referencias, duplicados, imágenes legibles, fuentes, ausencia de datos privados y criterios de aprobación.
linguanova_next/packaging/ native_export.py	Escribe traducciones nativas compatibles o llama a emisores recibidos que satisfagan el contrato. Mantiene IDs y evita duplicar old por idioma.
linguanova_next/packaging/ staging.py	Gestiona copias de salida y publicación del paquete validado. Un fallo no sobrescribe el original ni la última salida válida.

Salida del juego	Regla
game/tl/<renpy_id>/	Archivos de texto y recursos por destino. Para español, carpeta propia por perfil nuevo; spanish anterior se conserva.
zzz_linguanova_preferences_languages.rpy	Auxiliar nuevo, generado desde código nuevo: opciones solo en Preferencias, sin atajos, panel externo ni botón flotante, y elección persistente.
zzz_linguanova_ocultar_fuentes.rpy	Protección automática recibida, conservada sin cambios. No añade un selector de idioma ni exige copiar un parche manual.
game/linguanova_fonts/ y rutas antiguas	Conservar ubicación nueva fuera de tl y compatibilidad con recursos anteriores. No eliminar fuentes para ocultar un falso idioma.
Informes del creador	coverage_report y pendientes se entregan separados. El jugador no requiere OCR, modelos ni LinguaNova.

Compatibilidad del paquete

Ren’Py permite seleccionar idiomas y usar recursos localizados bajo tl/<idioma>/ respetando su ruta relativa [W1]. Comprobar cambios de idioma, recursos precargados y estilos en juegos reales. Instalar archivos nuevos puede requerir reiniciar el juego; cambiar entre idiomas ya cargados conserva la capacidad en caliente recibida.

El exportador detecta duplicados entre MOD y traducción completa y auxiliares anteriores con atajos. No publica dos dueños de la misma cadena. Una migración solo trata archivos generados identificados por su manifiesto, con respaldo en la copia; los originales y las correcciones ajenas permanecen intactos. Los casos de propiedad incierta quedan pendientes.

14 · PRUEBAS PARA LOS CINCO IDIOMAS

Conservar regresiones y añadir evidencia nueva

La documentación incluida declara 243 pruebas aprobadas, 12 omitidas y 140 cambios dirigidos en siete juegos de prueba con Ren’Py 8.3.7.25031702. Son resultados históricos entregados con la base; no se han repetido durante esta preparación. Los 140 cambios no son 140 traducciones reales de proveedores.

Prueba nueva bajo tests/next/	Verificación
test_base_integrity.py	Los 283 archivos conservan hash, ruta y tamaño; cero borrados, reemplazos o renombrados.
test_five_destination_registry.py	Se habilitan exactamente es, fr, pt, ru y en. Las variantes de español no agregan entradas base ni destinos futuros.
test_multidioma_preservation.py	Lista acumulativa, Original, cachés, JSON, fuente rusa y protección automática conservados; Preferencias sin atajos ni controles duplicados del MOD.
test_spanish_profile_isolation.py	Perfil en todas las solicitudes, control de mezcla y separación de cachés entre LATAM, España y datos heredados.
test_coverage_audit_readonly.py	Auditoría sin motores, mutaciones, alias ni escritura de caché; categorías reconciliadas por entrada.
test_ocr_bridge_contract.py	Destino compartido, origen independiente, respuesta correlacionada, cambio de selector y rechazo de respuestas tardías.
test_image_and_package.py	Inventario, incertidumbre, máscaras, composición, duplicados, fuentes y exportación sin diagnósticos.
test_future_catalog_contract.py	Fixture sintético ampliado para probar capacidad estructural sin habilitar idiomas reales nuevos ni afirmar soporte lingüístico.

Matriz de pruebas actual

Para cada destino habilitado, comprobar transmisión de código, caché, escritura, exportación y representación. Separar pruebas de proveedor simulado, proveedor real y juego real. Si se prueban cambios entre los cinco idiomas instalados, son 20 direcciones distintas más los retornos a Original; eso no demuestra las parejas de traducción de cada motor.

No modificar tests históricos para ocultar un fallo ni tratar como actual un manifiesto antiguo que espera otros locales. Los resultados iniciales se conservan para comparar cada fase.

15 · ORDEN DE APLICACIÓN

Implementar por fases sin editar la base

Fase	Archivos a incorporar y aplicación	Condición para avanzar
A. Congelar	baseline.py, manifiesto inicial y test_base_integrity. Copiar ZIP y juego; registrar versión y hashes.	283 archivos idénticos y resultados iniciales documentados.
B. Arranque	run_next.py, bootstrap, contracts, settings y panel complementario. Acceso al modo clásico.	Inicio separado; no hay dos sesiones escribiendo sobre la misma copia.
C. Cinco destinos	language_registry, languages.json con cinco entradas, perfiles del español y adaptadores de configuración/proveedor.	Español, francés, portugués, ruso e inglés; perfiles persistidos sin tocar constantes antiguas.
D. Texto y auditoría	Adaptador de extracción, clasificación, result_store, coverage_audit y residual_review.	Pendientes localizados, métricas separadas, auditoría sin mutaciones.
E. Puente y OCR	ocr_messages, ocr_translation_bridge, OCR e inventario. Primero proveedores simulados.	Contrato estable; el destino elegido llega a todas las regiones.
F. Imagen	Restauración clásica, traducción, composición y validación. IA generativa opcional después.	Original recuperable, resultados legibles y variante revisada.
G. Salida	Exportador nuevo de Preferencias, persistencia y paquete sobre staging; reutilizar solo contratos compatibles.	Interfaz nativa sin atajos y elección conservada tras reiniciar; lo no comprobado queda pendiente.
H. Distribución	Compilación e instalador nuevos, pruebas de actualización y recuperación.	Instalación limpia y actualización sin perder datos o funciones.

Cómo aplicar cada incremento

Añadir solo los archivos de la fase, ejecutar verificaciones de integridad y pruebas relevantes, guardar evidencia y revisar los resultados reales. Después habilitar la función en el arranque nuevo. La fase se recupera desactivando el componente nuevo o volviendo a la versión validada, sin restaurar archivos base que nunca fueron editados.

Las APIs o proveedores incompatibles se resuelven con adaptadores o implementación nuevos. No se cambia su contrato mediante sustitución de funciones. Las dependencias opcionales se agregan en archivos de requisitos nuevos y no se descargan modelos durante una importación.

16 · DISTRIBUCIÓN Y ACTUALIZACIÓN

El mismo proyecto con versión nueva propia

Archivo nuevo propuesto	Responsabilidad / contrato
linguanova_next/ version.py	Versión de la ampliación, esquemas y versión mínima de base. La versión 1.1.2 recibida se conserva como procedencia.
requirements-next.txt	Dependencias de la ampliación. No cambia requirements.txt; separa extras de OCR y generación local cuando se implementen.
LinguaNova_next.spec	Compila el arranque nuevo y los recursos requeridos. Usa los fuentes multidioma 1.1.2 intactos, sin sustituirlos por la base anterior.
build_next.bat	Construcción de la ruta nueva con los requisitos y el spec nuevos; no modifica build_windows.bat.
installer/ LinguaNova_next.iss	Instalador nuevo, con contrato de actualización y continuidad de identidad que se deben comprobar en una instalación anterior.
.github/workflows/ release-next.yml	Automatización añadida con disparador independiente del v* recibido, evitando dos compilaciones que publiquen activos incompatibles.
docs/ INTEGRACION_NEXT.md	Guía de arranque, instalación, límites, datos y recuperación. No reemplaza los informes de verificación multidioma recibidos.

Contrato de actualización conservado

El repositorio configurado sigue siendo Mustadio7813/LinguaNova. El actualizador instalado de Windows reconoce LinguaNova_Setup_<versión>.exe y SHA256SUMS.txt. Mantener esos activos y validar el comparador de versiones; no afirmar actualización automática para formatos que la base solo deriva a descarga.

La nueva versión se obtiene de su archivo propio. El modo clásico puede mostrar 1.1.2, y la ruta ampliada la versión del conjunto. Probar que abrir el modo clásico no produzca bucles de actualización ni sobrescriba la configuración nueva.

Recursos y modelos

PyInstaller admite declarar datos y módulos en el spec [W2]. La compilación nueva debe incluir catálogos, perfiles, QML y dependencias necesarias. Los pesos opcionales se distribuyen aparte con compatibilidad y hashes. Ninguna automatización ni publicación se ha ejecutado como parte de este informe.

17 · AMPLIACIÓN FUTURA

Preparar 139 sin implementarlos ahora

La etapa actual termina con cinco destinos habilitados. La ampliación a 139 es un proyecto posterior, apoyado en los contratos de esta base. Preparar el esquema no equivale a admitir ni probar esos 139 idiomas.

Preparar ahora	Trabajo posterior al ampliar
Catálogo externo y versionado	Añadir registros comprobados, sin modificar el coordinador para cada idioma.
Identidades separadas	Resolver códigos de proveedor, carpeta Ren'Py, escritura y variantes de cada destino añadido.
Interfaces de capacidades	Comprobar origen/destino, modelos y límites reales. Bloquear lo no compatible con motivo visible.
Cachés sin claves ambiguas	Mantener separación por locale, contexto, modelo y reglas en todos los índices y coincidencias.
OCR independiente del destino	Agregar reconocedores por escritura de origen; preparar composición y fuentes de destino por separado.
Estados y métricas extensibles	Mantener el mismo significado de pendientes y validaciones; no convertir soporte desconocido en éxito.
Preferencias con lista acumulativa	Medir usabilidad con muchas traducciones, conservando Original, recursos ocultos, cachés limpias y elección persistente, sin añadir atajos.
Pruebas de contrato	Añadir pruebas lingüísticas, de fuentes, RTL, CJK y representación para cada nuevo grupo de escrituras.

Qué queda fuera de esta etapa

No se agregan 134 destinos, modelos de todas las escrituras ni carpetas vacías. No se publican listas de compatibilidad no comprobadas. No se cambia la interfaz a 139 idiomas ni se reescriben los filtros del MOD.

Escalado de combinaciones

139 destinos no garantizan todos los pares de traducción. Si también fueran 139 orígenes y todas las parejas distintas fueran válidas, habría 19.182 direcciones; se crean tareas solo bajo demanda. Las pruebas sintéticas de tamaño miden contratos y rendimiento, no calidad de traducción.

18 · ACEPTACIÓN DEL PROYECTO

Condiciones concretas de entrega

Requisito	Evidencia necesaria
Base intacta	Manifiesto de los 283 archivos y comparación final: cero discrepancias. ZIP y copia de referencia idénticos.
Cinco destinos actuales	Selector con español, francés, portugués, ruso e inglés; español con perfil separado. Mantener los idiomas que el juego ya tenga instalados.
Mejoras multidioma	Lista acumulativa, Original, cachés, fuentes rusas, filtro automático y convivencia con MOD conservados; los atajos heredados no se generan en la ruta nueva.
Interfaz y persistencia	Idiomas solo en Preferencias o su equivalente, con el estilo del juego, sin atajos ni panel externo. La elección sobrevive a cierre y reapertura en una prueba real.
Código nuevo aplicado	Lista de archivos añadidos, arranque propio y dependencias explícitas; sin parches manuales ni sustitución de métodos.
Variante lingüística	Perfil transmitido y comprobado en texto e imágenes. Separación de datos heredados y aprobación identificada del material exportado.
Puente OCR exclusivo	Mensajes correlacionados, destino inmutable y pruebas de origen distinto, cancelación y respuestas tardías.
Porcentaje explicable	Listado de cada resto con causa y ubicación. Bloqueos y copias reconciliados sin doble conteo; métricas y universos visibles.
Paquete correcto	Archivos legibles y referencias válidas, sin duplicados, diagnósticos, credenciales o recursos expuestos como idiomas.
Futuro 139 previsto	Esquema y contratos extensibles con pruebas estructurales; ninguna afirmación de soporte lingüístico antes de validarlo.

Contenido de la entrega futura del desarrollador

Proyecto completo con la base conservada, archivos nuevos y sus funciones; instrucciones para ambos arranques; resultados iniciales y posteriores por fase; esquemas de datos; matriz de capacidades de los cinco destinos; muestras y limitaciones; paquete de prueba y procedimiento de recuperación.

Límites que deben declararse

La integridad por hashes y la correlación del destino son verificables. La detección automática de todo regionalismo, todo texto en imágenes o todas las rutas del juego no es infalible. La revisión lingüística y las pruebas reales deben figurar con su alcance, sin atribuirlas a fixtures o pruebas históricas.

19 · PROCEDENCIA Y REFERENCIAS

Base bloqueada y encargo técnico final

Base autorizada: LinguaNova_V2.2_Multidioma..zip. Carpeta interna: LinguaNova_Multidioma. Versión interna: 1.1.2. Tamaño: 1.433.406 bytes. Contiene 283 archivos.

SHA-256 del ZIP: `d702b683002f7cebccb92fb582c4b7ac2021b2ab0100cac41106470aaa558747`

Huellas de componentes protegidos

src/core/language_menu_template.py

`49ea64a2369b44fe15439c29c4b2ee72eaa39990934d1645f6851b107ad0ef7f`

src/core/pipeline/saving.py

`df11122862f53d1f6e0f9d57f04fdf43b8445a1108a3c58c2bb086bbfff2b08f`

src/core/runtime_hook_template.py

`cb112d59634f56a9ee9a418565963ccaaaeeb252af91e6a33d3d5457e67dd9fc`

src/tools/font_injector.py

`2d8d9ad01b163b797bbc7594b46fd657a3b2d00851a0ce9d7e0f448a995f485e`

src/ui_mod/generator.py

`703ae97ea9dbeac410a1a4733566f8042221400de1af40016c5b9f77c3d8f6e5`

El manifiesto completo de la implementación deberá cubrir los 283 archivos; estas huellas destacadas no sustituyen ese control. Durante la preparación de este informe se comparó la copia con los bytes del ZIP, sin editar los fuentes.

Referencias de diseño

[B1] Archivo adjunto, LEEME_MULTIDIOMA.txt, docs/VERIFICACION_MULTIDIOMA.md y docs/RESULTADOS_MULTIDIOMA.json. Sus pruebas declaradas son antecedentes, no ejecuciones nuevas de esta entrega.

[B2] Informe previo de implementación modular y requisitos de la conversación: copias, código nuevo, español sin mezcla y puente OCR / traductor. La aclaración posterior fija cinco destinos actuales y 139 solo a futuro.

[W1] Ren’Py, Translation: https://www.renpy.org/doc/html/translation.html. Consulta: 21-10-2026.

[W2] PyInstaller, Using Spec Files: https://pyinstaller.org/en/stable/spec-files.html. Consulta: 21-10-2026.

[W3] Ren’Py, Persistent Data: https://www.renpy.org/doc/html/persistent.html. Consulta: 21-10-2026.

Encargo para aplicar este informe

Trabajar sobre una copia de la base 1.1.2 y conservar sus archivos. Añadir código nuevo para cinco destinos, con cambio de idioma solo desde Preferencias, sin atajos y con elección persistente. Conservar las mejoras compatibles y no generar los selectores heredados con teclas. Preparar contratos para 139 solo a futuro.

ANEXO A · PANEL COMPLEMENTARIO

Archivos de interfaz y control de sesión

El panel se añade mediante código nuevo. No edita Main.qml, AppBackend ni los cinco locales recibidos. Su aspecto puede seguir las convenciones visuales de LinguaNova, pero sus acciones pertenecen al arranque nuevo.

Archivo nuevo propuesto	Responsabilidad / contrato
linguanova_next/ui/ TranslationPanel.qml	Vista principal nueva: cinco destinos, selector de perfil para español, opciones de texto e imágenes y resumen del trabajo antes de empezar.
linguanova_next/ui/ backend.py	Expone listDestinations(), listSpanishProfiles(), createJob(), startJob() y cancelJob(); publica eventos del contrato sin sustituir métodos de AppBackend.
linguanova_next/ui/ OcrStatusPane.qml	Muestra regiones, origen reconocido, candidata, máscara y composición. Lee el destino del trabajo; no contiene un selector OCR de destino independiente.
linguanova_next/ui/ ReviewPanel.qml	Presenta entrada, contexto y causa. Envía correcciones y decisiones por ID al flujo de validación; no modifica directamente el juego.
linguanova_next/storage/ settings_store.py	Persiste settings_next.json con esquema propio. Importa solo una copia explícita de los datos compatibles de la configuración anterior.
linguanova_next/core/ job_lock.py	Evita trabajos simultáneos sobre la misma copia y coordina las sesiones abiertas desde el lanzador nuevo. No controla por sustitución la cola clásica.

Campos mínimos y autoridad del destino

target_language_id ofrece es, fr, pt, ru y en. spanish_profile solo aparece al seleccionar español y fija el locale. project_path, source_mode, provider_id y image_enabled completan la solicitud. El origen estimado por OCR es un dato de reconocimiento; no cambia target_locale.

Orden al pulsar Iniciar

El backend valida la base y la ruta de copia, resuelve destino y perfil, crea el contrato inmutable y adquiere el bloqueo. Después inicia texto y/o imagen. El puente recibe ese mismo contrato. Al terminar, libera la sesión y presenta por separado actividad, cobertura y pendientes.

El botón no se habilita para un destino futuro no registrado o una capacidad incompatible. Cambiar la selección no altera un trabajo activo. Este panel pertenece a LinguaNova y no es el menú que se incorpora al juego. Las pruebas deben verificar estas reglas con los cinco destinos antes de ampliar el catálogo.

ANEXO B · INTEGRACIÓN NATIVA OBLIGATORIA

Los idiomas deben parecer parte del juego

Requisito de aceptación: idiomas integrados únicamente en Preferencias o el apartado equivalente del juego, con sus estilos y navegación. No generar teclas de idioma, botones flotantes ni paneles externos. Guardar la elección y restaurarla al volver a abrir.

Archivo nuevo propuesto	Responsabilidad / contrato
linguanova_next/core/ native_ui_policy.py	require_preferences_only() exige ubicación en Preferencias, ausencia de atajos y persistencia comprobada. Una integración incompleta bloquea la publicación como paquete completo.
linguanova_next/packaging/ native_ui_export.py	Genera desde código nuevo un auxiliar exclusivo de Preferencias con perfil del juego verificado. Reutiliza contratos compatibles sin invocar el selector heredado con teclas ni editar screens.rpy.
linguanova_next/core/ native_ui_audit.py	Revisa estilo, navegación, ubicación, estados visuales y ausencia de atajos o paneles de idioma generados. Registra incertidumbre y revisión visual.
linguanova_next/data/game_ui_profiles/ <game_id>.json	Receta específica, solo tras inspeccionar el juego: pantalla, contenedor, estilos y controles reconocidos. Un diseño desconocido no se adivina como si fuera compatible.
tests/next/ test_native_game_ui.py	Cinco destinos en Preferencias, controles originales, retorno a Original, persistencia, recursos coherentes y cero atajos o paneles externos añadidos.

Qué debe mantenerse en la presentación

Fondos, contenedores, colores, estilos de botones, iconos, navegación y acciones de guardar, cargar, historial y preferencias. El contenido puede adaptarse al espacio del idioma sin rediseñar la interfaz. Las imágenes de botones conservan sus estados normal, hover y seleccionado. Las fuentes originales se mantienen cuando cubren los caracteres; si falta cirílico, se usa soporte compatible con el estilo y se revisa el resultado.

Tratamiento de menús personalizados

Si no se reconoce Preferencias, Ajustes, Opciones o su equivalente, se crea una receta verificada para ese juego. No se resuelve el caso generando Shift+L. Los exportadores heredados con teclas quedan fuera de la salida nueva, incluido el selector autónomo del MOD; los fuentes originales se conservan. El paquete nuevo utiliza su salida nativa sin UI duplicada.

Sin la carpeta y ejecución del juego no puede certificarse su apariencia. Revisar los cinco destinos en la copia real. Una duda de interfaz o persistencia queda como condición pendiente del paquete y no invalida el texto ya aprobado. Los atajos originales del desarrollador ajenos al cambio de idioma se conservan.

ANEXO C · PERSISTENCIA Y SIN ATAJOS

Elegir una vez y conservar al reiniciar

Archivo nuevo propuesto	Responsabilidad / contrato
linguanova_next/packaging/ preferences_template.py	Emite solo el grupo de idiomas dentro de la pantalla original de Preferencias, con estilos heredados. No emite key, overlays ni acciones de apertura de un panel de idiomas.
linguanova_next/packaging/ persistence_template.py	Emite la lógica nueva de selección y guardado con APIs de Ren’Py. Distingue Original elegido de ausencia de elección; conserva también el perfil de español.
linguanova_next/packaging/ managed_output_migration.py	Detecta auxiliares anteriores que generen atajos. Migra únicamente salidas identificadas en staging, con respaldo y manifiesto; no edita fuentes originales ni archivos ajenos.
tests/next/ test_language_preference_persistence.py	Seleccionar, guardar, cerrar el proceso y abrir uno nuevo. Probar los cinco idiomas, perfiles de español y Original, sin depender de una partida guardada.

Flujo de selección

Desde Preferencias, una acción nativa Language() o una llamada compatible a renpy.change_language() aplica la elección. El auxiliar guarda el identificador y, cuando corresponda, el perfil, en la persistencia del juego. Usa tipos simples y un campo propio, por ejemplo persistent.linguanova_language_choice, sin escribir directamente _preferences.language [W1].

Ren’Py carga los datos persistentes al iniciar y permite guardarlos con renpy.save_persistent() [W3]. Guardar al confirmar la selección, sin esperar una partida guardada o la salida del juego. Probar la secuencia de acciones en la versión usada; informar un fallo en Preferencias. Conservar el directorio de datos del juego y no limpiar su persistencia al actualizar.

Arranque y prioridad

La elección guardada debe prevalecer sobre el último idioma generado por LinguaNova y sobre el destino elegido en el programa creador. No se fuerza un idioma fijo en cada inicio. Inspeccionar config.language y rutinas de arranque: pueden sobreponerse a la preferencia recordada [W1]. Un conflicto debe resolverse con el auxiliar nuevo y probarse; no se declara persistencia antes de esa verificación.

Aceptación final

Seleccionar ruso desde Preferencias, cerrar y abrir: sigue en ruso. Repetir con los otros cuatro idiomas y los perfiles españoles. Elegir Original y reiniciar: sigue Original. Verificar diálogos, menús, imágenes, fuentes y pantalla de preferencias. Cero teclas de cambio de idioma ni controles externos generados; los datos de la base continúan intactos.